4.4 Poisson regression

Vladimir Buskin

English Language and Linguistics, Catholic University of Eichstätt-Ingolstadt

ORCID: 0009-0005-5824-1012

Last updated: 14 October 2026, 22:04

This handout introduces Poisson regression for modelling count data, covering the Poisson distribution, the log link, the interpretation of coefficients as multiplicative effects, overdispersion and negative binomial regression as well as offsets for corpus frequencies, using sensory ratings and frequencies of English adjectives.

Table of contents

Recommended reading . . . 1
Preparation . . . 2
Why not linear regression? . . . 2
The Poisson distribution . . . 3
Poisson regression . . . 3
    Application in R . . . 4
    Interpretation . . . 7
Overdispersion . . . 7
    Negative binomial regression . . . 8
Counts from texts of different sizes: Offsets . . . 10
Exercises . . . 11
References . . . 11

Recommended reading

Winter (2020: Chapter 13)

Baguley (2012: Chapter 17.5)

Gelman & Hill (2007: Chapter 6.2)

Preparation

```
# Load libraries
library(tidyverse)
library(ggeffects)
library(readxl)

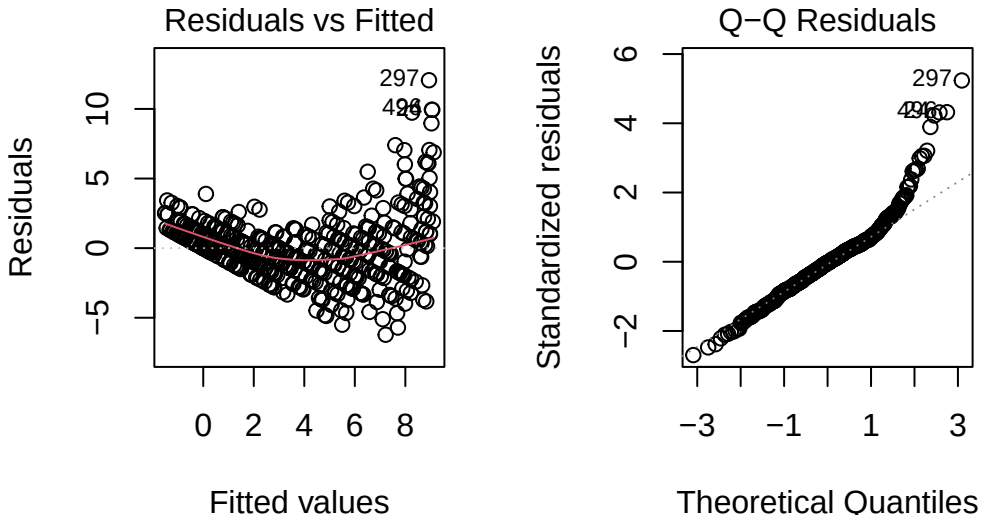
# Load datasets
adjectives <- read_xlsx("winter_2020_visual.xlsx")
```

Why not linear regression?

Frequency data is ubiquitous in corpus linguistics. Given its numeric nature, it seems tempting to model such data using linear regression, but doing so is bound to cause problems. Counts are **non-negative integers** (0, 1, 2, …), whereas the linear model assumes a continuous, normally distributed response that can, in principle, take any value. As a consequence,

- the linear model may predict **negative** counts, which are impossible,
- the residuals tend to be **non-normal**, and
- the residuals display **non-constant variance**: The larger the counts, the more they vary.

The figures below illustrate these issues for a linear model fitted to simulated count data. Note the fan-shaped spread of the residuals on the left and their deviation from the diagonal line on the right.

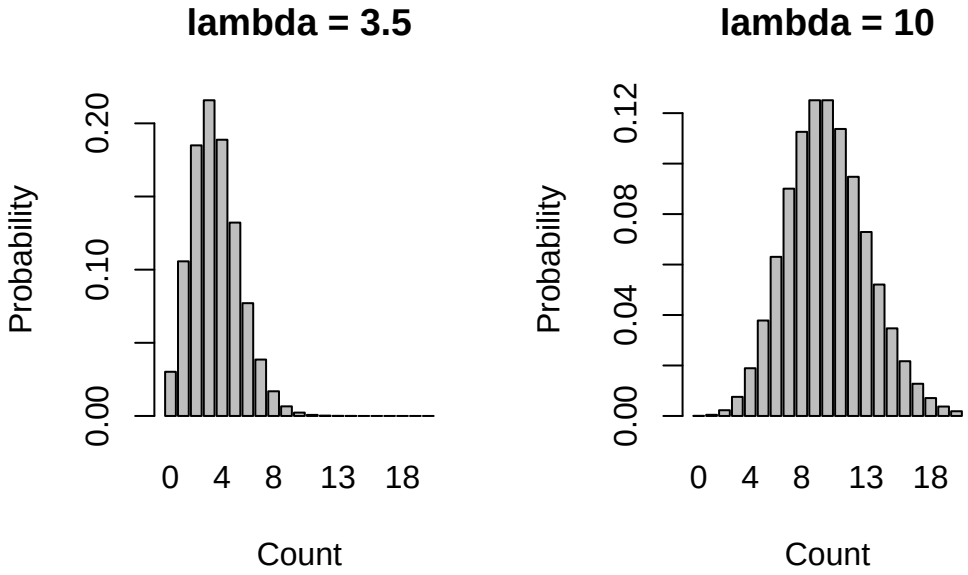


The Poisson distribution

A probability distribution that is much better equipped for this special case of discrete, yet numeric data is the **Poisson distribution**. Assuming a Poisson-distributed variable X, its exact shape is determined by a single parameter, λ ('lambda'), which is both its expected value (i.e., the mean) and its variance. In shorthand notation,

$$X \sim Pois(\lambda), \qquad E(X) = Var(X) = \lambda. \tag{1}$$

Its probability mass function has a notable **positive** (right) skew: Most of the probability mass lies on small counts, with a long tail stretching towards larger counts. The skew becomes less conspicuous for increasingly higher parameter values.



> **i** The Poisson probability mass function
>
> The probability of observing exactly k events (e.g., k occurrences of a word) is
>
> $$P(X = k) = \frac{\lambda^k e^{-\lambda}}{k!}, \qquad k = 0, 1, 2, ... \tag{2}$$
>
> where $k! = k \cdot (k-1) \cdot ... \cdot 1$ (and $0! = 1$). For instance, if a word occurs on average $\lambda = 3.5$ times per text, the probability that it does not occur at all in a given text is $P(X = 0) = e^{-3.5} \approx 0.03$. In R, these probabilities are computed by `dpois()`, e.g., `dpois(0, lambda = 3.5)`.

Poisson regression

Recall how in linear regression, the dependent variable Y was modelled as a function of the linear sum of predictor terms $\beta_p X_p$ for $1, ..., p$ independent variables.

The **Poisson model** keeps this linear predictor but links it to the response in a different way. It assumes that each observed count Y comes from a Poisson distribution whose expected value λ depends on the predictors, and it models the **logarithm** of λ:

$$Y \sim Pois(\lambda), \qquad \ln(\lambda) = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + ... + \beta_p X_p. \tag{3}$$

Two aspects of this model are worth noting:

- It is the expected count λ that is log-transformed, not the observed counts Y themselves. This is fortunate, since $\ln(0)$ is undefined, but zero counts are perfectly common. The logarithm is the **link function** of the model, much like the logit in logistic regression, and it ensures that the predicted counts are always positive.
- There is no separate error term. The random variation of the counts around λ is described by the Poisson distribution itself.

To remove the logarithm and obtain the model output on a more intuitive scale, we simply exponentiate both sides:

$$\lambda = e^{\beta_0 + \beta_1 X_1 + \beta_2 X_2 + ... + \beta_p X_p} = e^{\beta_0} \cdot e^{\beta_1 X_1} \cdot e^{\beta_2 X_2} \cdot ... \cdot e^{\beta_p X_p}. \tag{4}$$

The right-hand side shows that the predictors do not *add* to the expected count, but **multiply** it. This is key to interpreting the model output.

Application in R

The data provided by Winter (2020) contains frequency data for several hundred English adjectives (`Freq` column) as well as ratings of how strongly they are experienced through each of the five senses (`Sight`, `Touch`, `Sound`, `Taste` and `Smell`). The ratings range from 0 to 5 and were originally compiled by Lynott and Connell (2009).

```
head(adjectives)
```

```
# A tibble: 6 x 9
  Word      DominantModality Sight Touch Sound  Taste  Smell Log10Freq  Freq
  <chr>     <chr>            <dbl> <dbl> <dbl>  <dbl>  <dbl>     <dbl> <dbl>
1 abrasive  Haptic            2.89 3.68  1.68  0.579  0.579      1.36     23
2 absorbent Visual            4.14 3.14  0.714 0.476  0.476      0.903     8
3 aching    Haptic            2.05 3.67  0.667 0.0476 0.0952     2.02    105
4 acidic    Gustatory         2.19 1.14  0.476 4.19   2.90       1.04     11
5 acrid     Olfactory         1.12 0.625 0.375 3      3.5        0.301     2
6 adhesive  Haptic            3.67 4.33  1.19  0.905  1.76       1.67     47
```

We will examine the effects of the different sensory ratings on the frequency of an adjective. When fitting the generalised linear model with `glm()`, it is important to specify the argument `family = "poisson"`, which applies the logarithmic link function.

```
# Fit Poisson regression model
freq.m1 <- glm(Freq ~ Sight + Touch + Sound + Taste + Smell, data =
 ↪  adjectives, family = "poisson")

# Summarise model statistics
summary(freq.m1)
```

```
Call:
glm(formula = Freq ~ Sight + Touch + Sound + Taste + Smell, family = "poisson",
    data = adjectives)

Coefficients:
             Estimate Std. Error z value Pr(>|z|)
(Intercept)  3.1201624  0.0118785  262.67   <2e-16 ***
Sight        0.8726608  0.0024550  355.46   <2e-16 ***
Touch        0.1490792  0.0009801  152.10   <2e-16 ***
Sound        0.1692741  0.0011662  145.15   <2e-16 ***
Taste        0.0790246  0.0019106   41.36   <2e-16 ***
Smell       -0.0963918  0.0019746  -48.82   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for poisson family taken to be 1)

    Null deviance: 1890910  on 361  degrees of freedom
Residual deviance: 1633858  on 356  degrees of freedom
  (61 observations deleted due to missingness)
AIC: 1636345

Number of Fisher Scoring iterations: 7
```

As in logistic regression, the estimates are on the scale of the link function, i.e., they indicate changes in the **log** of the expected frequency. Note also that `glm()` silently removes all rows with missing values (see the line `observations deleted due to missingness`).

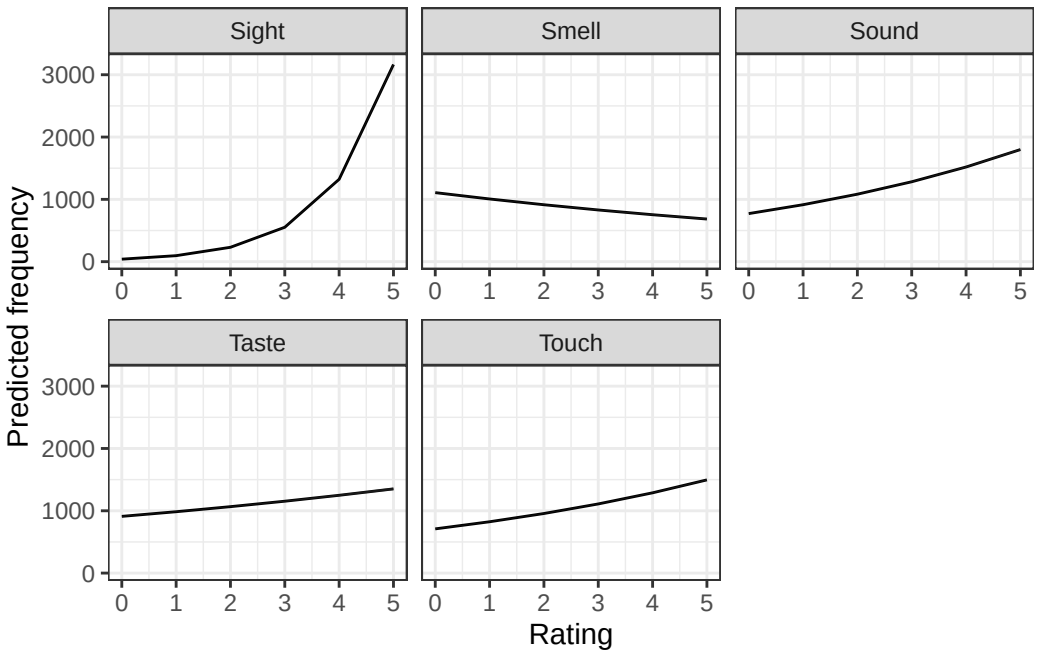
The model has identified numerous significant effects, i.e., β-values that are significantly different from 0. The standard errors are tiny, resulting in 95% confidence intervals that are barely visible in the effect plots below. The plots show the predicted frequencies for each rating, with all other predictors held constant at their mean values.

```
# Get predicted values for each predictor
pred_sight <- ggpredict(freq.m1, terms = "Sight")
pred_touch <- ggpredict(freq.m1, terms = "Touch")
pred_sound <- ggpredict(freq.m1, terms = "Sound")
pred_taste <- ggpredict(freq.m1, terms = "Taste")
pred_smell <- ggpredict(freq.m1, terms = "Smell")

# For plotting individual predictions, simply use:
# plot(pred_sight)

# Combine all predictions into one data frame
pred_all <- rbind(
  data.frame(pred_sight, predictor = "Sight"),
  data.frame(pred_touch, predictor = "Touch"),
  data.frame(pred_sound, predictor = "Sound"),
  data.frame(pred_taste, predictor = "Taste"),
  data.frame(pred_smell, predictor = "Smell")
)

# Create faceted plot
ggplot(pred_all, aes(x = x, y = predicted)) +
  geom_line() +
  geom_ribbon(aes(ymin = conf.low, ymax = conf.high), alpha = 0.5) +
  facet_wrap(~predictor, scales = "free_x") +
  labs(x = "Rating", y = "Predicted frequency") +
  theme_bw()
```



Interpretation

Since the predictors multiply the expected count (Equation 4), a one-unit increase in X changes the expected count by the factor e^{β}. Equivalently, the percentage increase (or decrease, respectively) is given by $100(e^{\beta} - 1)$.

As we can see, adjectives with higher `Sight` ratings (i.e., highly visual adjectives) are associated with the greatest increase in frequency of occurrence. The predictor `Sight` has an estimate of approximately 0.87 ($p < 0.001$, 95% CI [0.87, 0.88]), so each one-unit increase in the sight rating multiplies the expected frequency by $e^{0.87} \approx 2.39$. This corresponds to an increase of $100(e^{0.87} - 1) \approx 138.69\%$.

By contrast, `Smell` is associated with a lower frequency: Its estimate of -0.1 translates into a change of $100(e^{-0.1} - 1) \approx -9.52\%$ for each one-unit increase in smell ratings. In essence: Smelly adjectives are unpopular!

Or are they? Before we draw any conclusions, we need to check a crucial assumption of the model.

Overdispersion

According to Equation 1, the variance of a Poisson-distributed variable equals its mean. In practice, however, counts often vary much more than the Poisson model allows. This is called **overdispersion**, and it is the rule rather than the exception for linguistic frequency data: Some words are simply much more frequent than others in ways the predictors cannot capture.

Overdispersion does not bias the estimates themselves, but it causes the model to **underestimate the standard errors**. As a result, the p-values are too small and the confidence intervals too narrow. The tiny standard errors we observed above are therefore not a sign of robust estimates, but a warning sign.

A simple diagnostic is the **dispersion ratio**, i.e., the sum of the squared Pearson residuals divided by the residual degrees of freedom. If the Poisson assumption holds, it should be close to 1.

```
# Dispersion ratio
dispersion <- sum(residuals(freq.m1, type = "pearson")^2) /
 ↪  df.residual(freq.m1)

dispersion
```

```
[1] 13380.68
```

There is no universally agreed threshold, but values well above 1 indicate overdispersion. A ratio of about 1.3381×10^4 means that the residual variance is roughly 1.3381×10^4 times larger than what the Poisson model assumes, so it is clearly inappropriate for these data.

Negative binomial regression

The most common remedy is **negative binomial regression**. It has the same log link and the same interpretation as the Poisson model, but it adds a dispersion parameter θ ('theta') that allows the variance to exceed the mean:

$$Var(Y) = \lambda + \frac{\lambda^2}{\theta}. \tag{5}$$

The smaller θ, the stronger the overdispersion; as θ grows very large, the model approaches the Poisson model. Negative binomial models can be fitted with `glm.nb()` from the `MASS` package, which comes pre-installed with R.

```
# Fit negative binomial regression model
# MASS:: accesses the function without loading the package,
# which would otherwise mask dplyr's select() function
freq.nb <- MASS::glm.nb(Freq ~ Sight + Touch + Sound + Taste + Smell, data
 ↪  = adjectives)

summary(freq.nb)
```

```
Call:
MASS::glm.nb(formula = Freq ~ Sight + Touch + Sound + Taste +
    Smell, data = adjectives, init.theta = 0.3420955584, link = log)

Coefficients:
            Estimate Std. Error z value Pr(>|z|)
(Intercept)  3.44597    0.47589   7.241 4.45e-13 ***
Sight        0.79536    0.09727   8.177 2.92e-16 ***
Touch        0.11745    0.06249   1.880  0.06017 .
Sound        0.16258    0.07022   2.315  0.02060 *
Taste        0.27700    0.09885   2.802  0.00507 **
Smell       -0.21349    0.10452  -2.043  0.04109 *
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for Negative Binomial(0.3421) family taken to be 1)

    Null deviance: 563.53  on 361  degrees of freedom
Residual deviance: 485.45  on 356  degrees of freedom
  (61 observations deleted due to missingness)
AIC: 5325.6

Number of Fisher Scoring iterations: 1
```

```
          Theta:  0.3421
      Std. Err.:  0.0204

 2 x log-likelihood:  -5311.6140
```

The output resembles that of the Poisson model, with the estimated θ reported at the bottom. Crucially, the standard errors are now considerably larger. Both models can be compared using the AIC (lower values indicate a better fit; see 4.2 Logistic regression):

```
AIC(freq.m1, freq.nb)
```

```
        df         AIC
freq.m1  6 1636344.798
freq.nb  7    5325.614
```

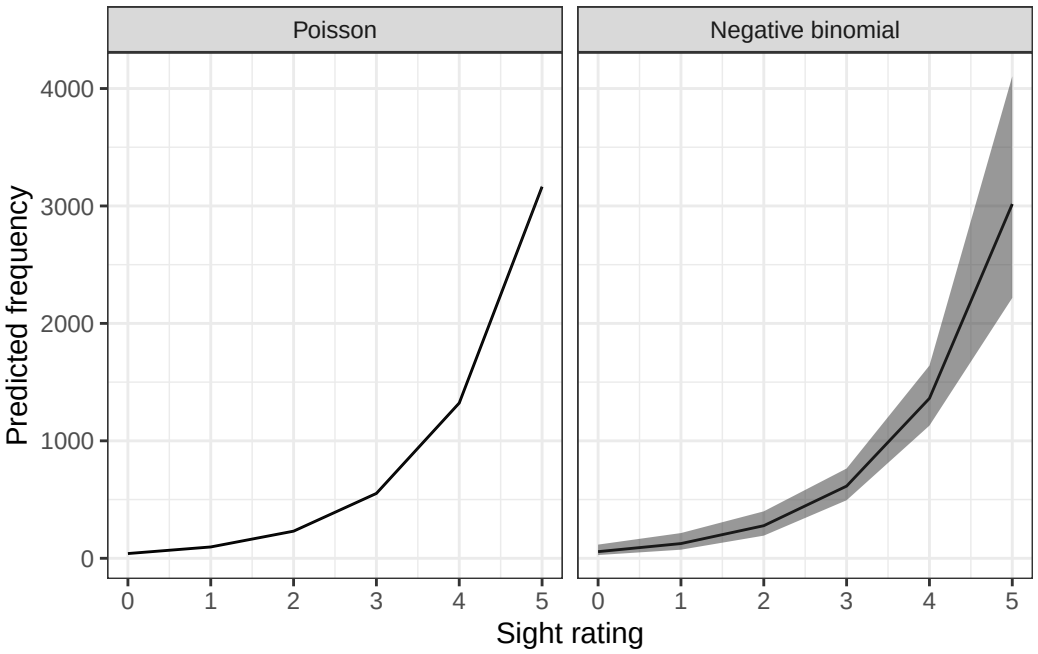
The difference in the confidence intervals becomes evident when we plot the predictions of both models for `Sight`:

```
pred_sight_nb <- ggpredict(freq.nb, terms = "Sight")

pred_comp <- rbind(
  data.frame(pred_sight, model = "Poisson"),
  data.frame(pred_sight_nb, model = "Negative binomial")
)

# Show the Poisson model first
pred_comp$model <- factor(pred_comp$model, levels = c("Poisson", "Negative
 ↪  binomial"))

ggplot(pred_comp, aes(x = x, y = predicted)) +
  geom_line() +
  geom_ribbon(aes(ymin = conf.low, ymax = conf.high), alpha = 0.5) +
  facet_wrap(~model) +
  labs(x = "Sight rating", y = "Predicted frequency") +
  theme_bw()
```



The coefficients are interpreted exactly as before, using $100(e^{\beta} - 1)$.

> **Quasi-Poisson models**
>
> An alternative to the negative binomial model is the **quasi-Poisson** model, which can be fitted with `glm(..., family = "quasipoisson")`. It produces the same estimates as the Poisson model, but multiplies the standard errors by $\sqrt{\text{dispersion ratio}}$. Since it does not rely on a full probability distribution, it does not provide an AIC.

Counts from texts of different sizes: Offsets

In corpus linguistics, counts are often obtained from texts or subcorpora of different sizes. A text with 10,000 words naturally contains more passives than one with 1,000 words, regardless of its register. Here, we are interested in **rates** (e.g., passives per word) rather than raw counts.

Poisson and negative binomial models handle this by means of an **offset**, i.e., the logarithm of the text size, which is added to the model with its coefficient fixed at 1:

$$\ln(\lambda) = \ln(\text{text size}) + \beta_0 + \beta_1 X_1 + ... + \beta_p X_p \quad \Leftrightarrow \quad \ln\left(\frac{\lambda}{\text{text size}}\right) = \beta_0 + \beta_1 X_1 + ... + \beta_p X_p \tag{6}$$

The model on the right is a model of the rate λ/text size. In R, we wrap the offset in `offset()`. For a hypothetical data frame `passive_data` with one row per text, the passive counts `n_passives`, the text sizes `n_words` and the `register` of each text, the model would look as follows:

```
passive.m <- MASS::glm.nb(n_passives ~ register + offset(log(n_words)),
 ↪  data = passive_data)
```

The exponentiated coefficients now indicate by which factor the **rate** of passives changes, e.g., from one register to another. This is preferable to modelling relative frequencies (e.g., passives per 1,000 words) with a linear model, since the counts retain the information about how much text each rate is based on.

Exercises

Exercise 1. Compute the percentage changes in the expected frequency for `Touch`, `Sound` and `Taste` in the Poisson model `freq.m1` and interpret them.

Exercise 2. Compare the output of the negative binomial model `freq.nb` with that of the Poisson model `freq.m1`. Which predictors still have a significant effect on frequency? How do the estimates and their standard errors differ? Does the conclusion that smelly adjectives are unpopular still hold?

Exercise 3. Fit a quasi-Poisson model to the adjective data and verify that its standard errors equal those of the Poisson model multiplied by $\sqrt{\text{dispersion ratio}}$.

Exercise 4. All frequencies in the adjective data were taken from the same corpus. Would it make sense to include an offset in the model? Why or why not?

References

Baguley, Thomas. 2012. *Serious Stats: A Guide to Advanced Statistics for the Behavioral Sciences*. Houndmills, Basingstoke: Palgrave Macmillan.

Gelman, Andrew, and Jennifer Hill. 2007. *Data Analysis Using Regression and Multilevel/Hierarchical Models*. Analytical Methods for Social Research. Cambridge: Cambridge University Press.

Lynott, Dermot, and Louise Connell. 2009. "Modality Exclusivity Norms for 423 Object Properties." *Behavior Research Methods* 41: 558–64. https://doi.org/10.3758/BRM.41.2.558.

Winter, Bodo. 2020. *Statistics for Linguists: An Introduction Using r*. New York; London: Routledge.